

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	Nikhil Kumar Reddy Kadiveti	Reg. No.:	192425031
Section / Batch:	2024	Date:	8/10/2026

Code of Conduct

I Nikhil Kumar Reddy Kadiveti (192425031) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.

Identify email addresses.

Extract mobile numbers.

Detect technical skills (Python, Java, SQL, Machine Learning, NLP).

Extract years of experience.

Generate a structured summary of the candidate's profile.

Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.

Perform prefix-based searches.

Perform suffix-based searches.

Search using partial keywords.

Perform case-insensitive searches.

Display all matching product names.

Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.

Validate the institutional email address.

Validate the course code.

Validate the semester information.

Validate the student's mobile number.

Display appropriate validation messages for each field.

Generate a final registration status report indicating whether the registration is successful.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1 – Regex-Based Resume Information Extraction

A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting. Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

- Extract the candidate's name.
- Identify email addresses.
- Extract mobile numbers.
- Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
- Extract years of experience.
- Generate a structured summary of the candidate's profile.
- Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Program

```
import re

resume = """
Name: Rahul Kumar
Email: rahul.kumar@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL, Machine Learning, NLP
Experience: 3 years
"""

name = re.search(r"Name:\s*([A-Za-z ]+)", resume)
email = re.search(r"[\w.-]+@[ \w.-]+\.\w+", resume)
mobile = re.search(r"\b[6-9]\d{9}\b", resume)
experience = re.search(r"Experience:\s*(\d+)\s*years?", resume, re.I)

skills_list = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
skills_found = []

for skill in skills_list:
    if re.search(r"\b" + re.escape(skill) + r"\b", resume, re.I):
        skills_found.append(skill)

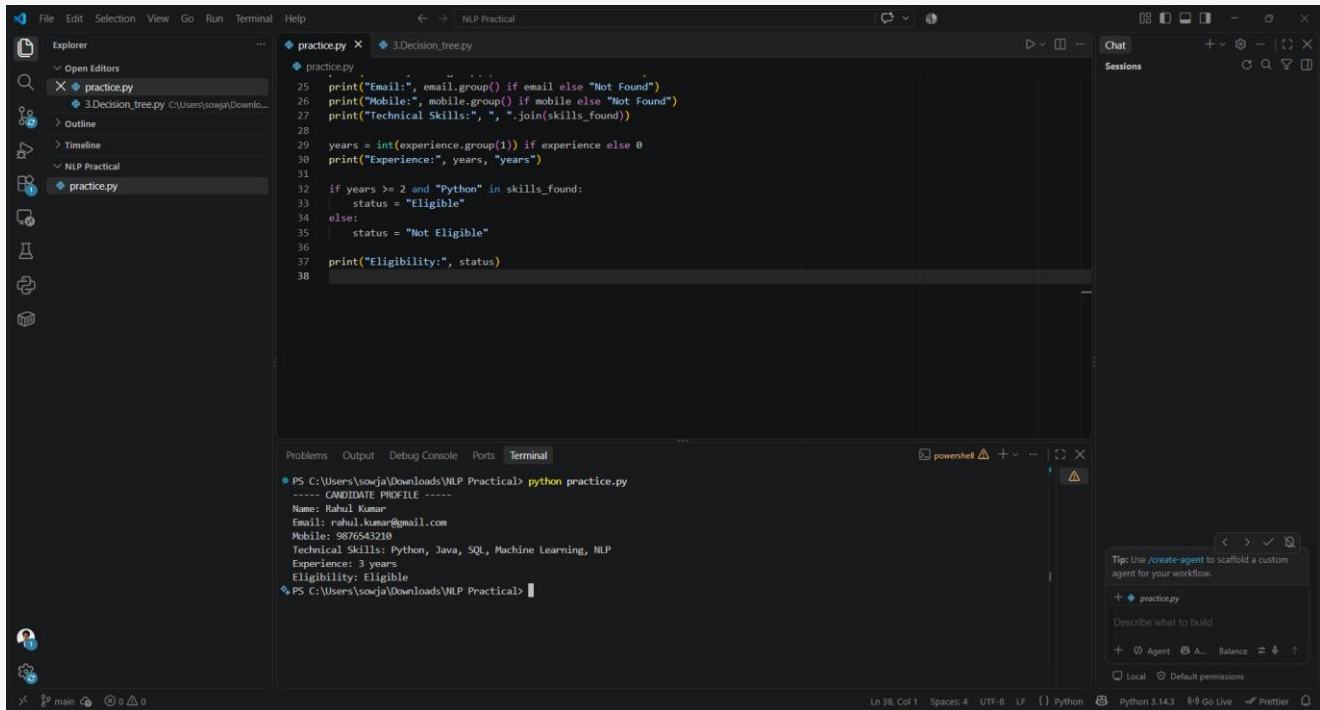
print("----- CANDIDATE PROFILE -----")
print("Name:", name.group(1) if name else "Not Found")
print("Email:", email.group() if email else "Not Found")
print("Mobile:", mobile.group() if mobile else "Not Found")
print("Technical Skills:", ", ".join(skills_found))

years = int(experience.group(1)) if experience else 0
print("Experience:", years, "years")

if years >= 2 and "Python" in skills_found:
    status = "Eligible"
else:
    status = "Not Eligible"
```

```
print("Eligibility:", status)
```

OUTPUT



The screenshot shows a Visual Studio Code editor window with a Python file named `practice.py` open. The file contains a script that prints candidate information and determines eligibility based on years of experience and technical skills. The terminal at the bottom shows the output of running `python practice.py`, which displays the candidate's profile and eligibility status.

```
25 print("Email:", email.group() if email else "Not Found")
26 print("Mobile:", mobile.group() if mobile else "Not Found")
27 print("Technical Skills:", ", ".join(skills_found))
28
29 years = int(experience.group(1)) if experience else 0
30 print("Experience:", years, "years")
31
32 if years >= 2 and "Python" in skills_found:
33     status = "Eligible"
34 else:
35     status = "Not Eligible"
36
37 print("Eligibility:", status)
38
```

Terminal Output:

```
PS C:\Users\sowja\Downloads\NLP Practical> python practice.py
----- CANDIDATE PROFILE -----
Name: Rahul Kumar
Email: rahul.kumar@gmail.com
Mobile: 9876543210
Technical Skills: Python, Java, SQL, Machine Learning, NLP
Experience: 3 years
Eligibility: Eligible
PS C:\Users\sowja\Downloads\NLP Practical>
```

Question 2 – Regex-Based Product Search System

An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

- Search products using an exact keyword.
- Perform prefix-based searches.
- Perform suffix-based searches.
- Search using partial keywords.
- Perform case-insensitive searches.
- Display all matching product names.
- Generate a report showing the total number of matching products for each search.

Program

```
import re

products = [
    "Python Programming Book",
    "Java Programming Book",
    "Python Laptop",
    "Java Laptop",
    "Machine Learning Course",
    "SQL Database Guide",
    "Python Course",
    "NLP Tutorial"
]

def search_products(keyword, search_type):
    if search_type == "exact":
        pattern = r"\b" + re.escape(keyword) + r"\b"
    elif search_type == "prefix":
        pattern = r"\b" + re.escape(keyword)
    elif search_type == "suffix":
        pattern = re.escape(keyword) + r"\b"
    elif search_type == "partial":
        pattern = re.escape(keyword)
    else:
        print("Invalid search type")
        return

    matches = []
    for product in products:
        if re.search(pattern, product, re.IGNORECASE):
            matches.append(product)

    print("\nSearch Keyword:", keyword)
    print("Search Type:", search_type)

    if matches:
        print("Matching Products:")
```

```

    for product in matches:
        print("-", product)
    else:
        print("No matching products found.")

print("Total Matching Products:", len(matches))

search_products("Python", "exact")
search_products("Python", "prefix")
search_products("Book", "suffix")
search_products("Prog", "partial")
search_products("python", "partial")

```

OUTPUT

The screenshot shows a VS Code editor with a file named `practice.py` open. The code in the editor is as follows:

```

30     matches.append(product)
31
32     print("\nSearch Keyword:", keyword)
33     print("Search Type:", search_type)
34
35     if matches:
36         print("Matching Products:")
37         for product in matches:
38             print("-", product)
39     else:
40         print("No matching products found.")
41
42     print("Total Matching Products:", len(matches))
43
44     search_products("Python", "exact")
45     search_products("Python", "prefix")
46     search_products("Book", "suffix")
47     search_products("Prog", "partial")
48     search_products("python", "partial")
49

```

The terminal output shows the execution of the script with the following results:

```

PS C:\Users\sowja\Downloads\NLP Practical> python practice.py
Search Type: partial
Matching Products:
- Python Programming Book
- Java Programming Book
Total Matching Products: 2

Search Keyword: python
Search Type: partial
Matching Products:
- Python Programming Book
- Python Laptop
- Python Course
Total Matching Products: 3
PS C:\Users\sowja\Downloads\NLP Practical>

```

Question 3 – Regex-Based University Registration Validation

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

- Validate the student register number.
- Validate the institutional email address.
- Validate the course code.
- Validate the semester information.
- Validate the student's mobile number.
- Display appropriate validation messages for each field.
- Generate a final registration status report indicating whether the registration is successful.

Program

```
import re

register_no = input("Enter Register Number: ")
email = input("Enter Institutional Email: ")
course_code = input("Enter Course Code: ")
semester = input("Enter Semester: ")
mobile = input("Enter Mobile Number: ")

register_pattern = r"^\d{9}$"
email_pattern = r"^\d{9}\.simats@saveetha\.com$"
course_pattern = r"^[A-Z]{3}\d{4}$"
semester_pattern = r"^[1-8]$"
mobile_pattern = r"^[1-9]\d{9}$"

register_valid = re.match(register_pattern, register_no)
email_valid = re.match(email_pattern, email)
course_valid = re.match(course_pattern, course_code)
semester_valid = re.match(semester_pattern, semester)
mobile_valid = re.match(mobile_pattern, mobile)

print("\n----- VALIDATION RESULTS -----")

print("Register Number:", "Valid" if register_valid else "Invalid")
print("Institutional Email:", "Valid" if email_valid else "Invalid")
print("Course Code:", "Valid" if course_valid else "Invalid")
print("Semester:", "Valid" if semester_valid else "Invalid")
print("Mobile Number:", "Valid" if mobile_valid else "Invalid")

if register_valid and email_valid and course_valid and semester_valid and mobile_valid:
    print("\nRegistration Status: SUCCESSFUL")
else:
    print("\nRegistration Status: FAILED")
```

OUTPUT

The image shows a Visual Studio Code editor window with a Python script named `practice.py` and its execution output in the terminal.

Script Content (`practice.py`):

```
1 import re
2
3 register_no = input("Enter Register Number: ")
4 email = input("Enter Institutional Email: ")
5 course_code = input("Enter Course Code: ")
6 semester = input("Enter Semester: ")
7 mobile = input("Enter Mobile Number: ")
8
9 register_pattern = r"^\d{9}$"
10 email_pattern = r"^\d{9}\.simats@saveetha\.com$"
11 course_pattern = r"^[A-Z]{3}\d{4}$"
12 semester_pattern = r"^[1-8]$"
13 mobile_pattern = r"^[1-9]\d{9}$"
14
15 register_valid = re.match(register_pattern, register_no)
16 email_valid = re.match(email_pattern, email)
17 course_valid = re.match(course_pattern, course_code)
18 semester_valid = re.match(semester_pattern, semester)
19 mobile_valid = re.match(mobile_pattern, mobile)
20
21 print("\n----- VALIDATION RESULTS -----")
22
23 print("Register Number:", "Valid" if register_valid else "Invalid")
24 print("Institutional Email:", "Valid" if email_valid else "Invalid")
25 print("Course Code:", "Valid" if course_valid else "Invalid")
```

Terminal Output:

```
PS C:\Users\souja\Downloads\NLP Practical> python practice.py
Enter Institutional Email: 192425031.simats@saveetha.com
Enter Course Code: DSA0304
Enter Semester: 5
Enter Mobile Number: 9618120426

----- VALIDATION RESULTS -----
Register Number: Valid
Institutional Email: Valid
Course Code: Valid
Semester: Valid
Mobile Number: Valid

Registration Status: SUCCESSFUL
PS C:\Users\souja\Downloads\NLP Practical>
```

The terminal output shows that all input fields (Register Number, Institutional Email, Course Code, Semester, and Mobile Number) were validated successfully, resulting in a "Registration Status: SUCCESSFUL" message.